Midterm Project Report

Advanced Computer Programming

Student Name	GIFT HLUZEKA MKHALIPHI
Student ID	114021104
Teacher	DINH-TRUNG VU
Date	2026-04-16

Made with GAMMA

Introduction

Github

- **Personal Github Account: https://github.com/ice632069-bytetab**
- **Group Project Repository: ACP-Group-4**

Overview

This project utilises the **Scrapy** framework to automate the collection of data from GitHub. The programme is designed to crawl a user's repository list and extract detailed metadata for each project.

Advanced language features and libraries used:

Python

The core language used for spider development and data logic.

Scrapy

A high-level crawling and web scraping framework used to handle asynchronous requests.

CSS Selectors

Implemented to navigate the HTML structure of GitHub to find repository names, counts, and languages.

XML Feed Export

The results are automatically serialised and saved into a structured XML file.

Results achieved: My programme successfully extracted information about the **URL, About, Language, and Number of Commits** for all repositories on my personal profile.

Implementation

Class 1: GithubRepoItem

Description: A data class that defines the fields required for the scraped information.

Fields: url, last_updated, about, languages, commits.

Class 2: ReposSpider

Description: The main scraping logic that handles navigation and extraction.

Method 1: parse

- Identifies each repository container on the main list page.
- Extracts the repository URL and Last Updated timestamp.
- Yields a follow-up request to the specific repository's detail page.

Method 2: parse_details

- **About Logic:** If the "About" section is empty, it extracts the repository name to use as the description.
- **Empty Repo Logic:** If the repository is empty (detected via the blankslate class), it sets languages and commits to None.
- **Metric Extraction:** Pulls the commit count using the selector `span.d-none.d-sm-inline strong`.

Results

Result 1: Execution Log

When the script runs, the terminal confirms the start and completion of the crawl, indicating that results are saved to github_repos.xml.

Result 2: Data Output

The programme generates a structured XML file where each repository is represented as an item containing the URL, About text, Languages used, and the total Number of Commits.

Conclusions

The project successfully demonstrates how to use Scrapy to handle dynamic web content and edge cases, such as empty repositories. By implementing specific logic for missing descriptions, the scraper ensures that the final XML output is complete and accurate for all profile repositories.

Made with GAMMA